

PostgreSQL pg_rewind

Post-Failover Recovery Runbook

Complete Step-by-Step Implementation Guide

Physical Replication: Log-Shipping (PG 13/14) and Streaming with Slots (PG 16/17)

Target Audience	PostgreSQL DBAs, DevOps Engineers, System Administrators
PostgreSQL Versions	13, 14 (Log-Shipping) 16, 17 (Streaming with Slots)
Data Directory	/apps/pgsql_data/<version>
Log Directory	/apps/logs
Operating System	AlmaLinux 8 / 9
Scope	Failover, pg_rewind, Standby Reconfiguration, Validation

Table of Contents

1. Executive Summary
2. Environment Overview and Architecture
3. Prerequisites and Requirements
4. Phase 1 – Initial Prerequisites Verification
5. Phase 2 – Execute Failover
6. Phase 3 – Prepare Old Primary for Reintegration
7. Phase 4 – Execute pg_rewind
8. Phase 5 – Configure Old Primary as New Standby
9. Phase 6 – Start and Verify Replication
10. Phase 7 – Cleanup and Post-Recovery Tasks
11. Troubleshooting Guide
12. Automation Script
13. Production Best Practices
14. Quick Reference Checklist

1. Executive Summary

This runbook provides a production-tested procedure for using PostgreSQL's `pg_rewind` utility to reintegrate a failed primary server into a replication cluster after a standby promotion. It covers **both** replication topologies in use across our environment:

LOG-SHIPPING (PG 13/14): PostgreSQL 13 and 14 — Physical replication via **WAL log-shipping** (archive-based). The primary archives WAL files to a shared location; the standby retrieves and replays them via `restore_command`. No direct streaming connection or replication slots are used.

STREAMING REPLICATION (PG 16/17): PostgreSQL 16 and 17 — Physical replication via **streaming replication with physical replication slots**. The standby maintains a persistent streaming connection to the primary and uses a replication slot to prevent WAL recycling.

Key Benefits of pg_rewind Over pg_basebackup

Factor	pg_rewind	pg_basebackup
Data transferred	Only changed blocks	Entire database
1 TB database, 10 GB divergence	5 – 15 minutes	10+ hours
Network bandwidth	Minimal	Full database size
Requires source running?	Yes (source-server mode)	Yes

Replication Method Quick Comparison

Aspect	Log-Shipping (PG 13/14)	Streaming + Slots (PG 16/17)
WAL delivery	Archive: primary writes WAL files to shared path; standby polls and replays	Streaming: standby pulls WAL records in real-time from primary
Replication lag	Up to <code>archive_timeout</code> (default 0 = only on WAL switch, can be minutes)	Sub-second (continuous stream)
Replication slots	Not used — WAL retention managed by archive	Physical slot prevents WAL recycling
Monitoring view	<code>pg_stat_archiver</code> (primary), file-level checks (standby)	<code>pg_stat_replication</code> (primary), <code>pg_stat_wal_receiver</code> (standby)
Network requirement	Shared filesystem or <code>rsync/scp</code> access to archive	TCP port 5432 between servers
pg_rewind difference	No slot to create; configure <code>restore_command</code> only	Must create replication slot; configure <code>primary_conninfo</code> + slot

Infrastructure Setup

NOTE: Throughout this runbook, \$PGDATA refers to /apps/pgsql_data/<version>. Replace <version> with the actual major version number (13, 14, 16, or 17) for your cluster. All commands assume you are logged in as the postgres OS user unless noted otherwise.

```

■ Primary Server ■ archive_command ■ Shared / NFS Archive ■
■ 192.168.1.10 ■ /apps/pgsql_archives/
■ /apps/pgsql_data/14 ■ (WAL files) ■
■ Read / Write ■
restore_command
■
▼
■ Standby Server ■
■ 192.168.1.20 ■
■ /apps/pgsql_data/14 ■
■ Read-Only ■

```

[illegible]

3. Prerequisites and Requirements

Critical Configuration (Must Be Set BEFORE Any Failover)

Parameter	Required Value	Purpose	Restart?
wal_log_hints	on	Enables pg_rewind to track block changes. Alternative: initdb --data-checksums	Yes
full_page_writes	on (default)	Complete page images in WAL after checkpoint — needed for crash-safe pg_rewind	Yes
wal_level	replica or logical	Must be at least replica for any physical replication	Yes
hot_standby	on (default PG 12+)	Allows read-only queries on standby for verification	Yes
archive_mode	on	Required for both log-shipping AND for restore_command during pg_rewind	Yes
archive_command	(see below)	Copies WAL to archive. Critical for log-shipping; also useful for streaming as fallback	No (reload)

CRITICAL: If `wal_log_hints` is OFF and data checksums were not enabled at `initdb` time, `pg_rewind` **cannot work**. You must fall back to `pg_basebackup`. Enable `wal_log_hints = on` proactively on ALL clusters.

LOG-SHIPPING (PG 13/14): Additional log-shipping requirements (PG 13/14): Both servers must have access to the WAL archive directory (`/apps/pgsql_archives`), either via shared NFS mount, `rsync`, or `scp`. The `archive_command` on the primary and `restore_command` on the standby must be configured and tested.

Log-Shipping Archive Commands (PG 13/14):

```
# Example archive_command on primary (PG 13/14 log-shipping)
archive_command = 'test ! -f /apps/pgsql_archives/%f && cp %p /apps/pgsql_archives/%f'

# Example restore_command on standby (PG 13/14 log-shipping)
restore_command = 'cp /apps/pgsql_archives/%f %p'
```

STREAMING REPLICATION (PG 16/17): Additional streaming requirements (PG 16/17): Firewall allows TCP port 5432 between servers. `pg_hba.conf` includes replication entries. A dedicated replication user with `REPLICATION` privilege exists. `max_wal_senders` ≥ 2 and `max_replication_slots` ≥ 2 .

pg_hba.conf Entries (PG 16/17):

```
# pg_hba.conf entries on both servers (PG 16/17 streaming)
host replication replicator 192.168.1.10/32 scram-sha-256
host replication replicator 192.168.1.20/32 scram-sha-256
# Also needed for pg_rewind source-server connection:
host all postgres 192.168.1.10/32 scram-sha-256
host all postgres 192.168.1.20/32 scram-sha-256
```

Recommended Safety Parameters (PG 13+)

Parameter	Recommended	Purpose
max_slot_wal_keep_size	50 GB (PG 16/17 only)	Prevents replication slots from filling disk if standby goes down
wal_keep_size	2 GB	Retains extra WAL as safety net (replaces wal_keep_segments in PG 13+)
archive_timeout	300 (log-shipping)	Forces WAL switch every 5 min even during low activity — reduces max data loss window for log-shipping

4. Phase 1 – Initial Prerequisites Verification

Purpose: Confirm replication health and verify all prerequisites before any failover.

Step 1: Verify Current Replication Status

On Primary Server (192.168.1.10) — All Versions:

```
-- Verify critical parameters (run on primary, all versions)
SELECT name, setting, pending_restart
FROM pg_settings
WHERE name IN ('wal_log_hints', 'full_page_writes', 'wal_level',
'hot_standby', 'archive_mode', 'archive_command',
'max_wal_senders', 'max_replication_slots',
'max_slot_wal_keep_size', 'wal_keep_size',
'archive_timeout');
```

LOG-SHIPING (PG 13/14): PG 13/14 — Verify log-shipping health: Check the archive is working. On the primary, query `pg_stat_archiver`. On the standby, verify WAL files are being restored.

Log-Shipping Verification (PG 13/14):

```
-- On PRIMARY (PG 13/14): Check archiver status
SELECT archived_count, last_archived_wal, last_archived_time,
failed_count, last_failed_wal, last_failed_time
FROM pg_stat_archiver;

-- Verify archive files exist
ls -ltr /apps/pgsql_archives/ | tail -5
```

Standby Verification (PG 13/14):

```
-- On STANDBY (PG 13/14): Verify recovery is running
SELECT pg_is_in_recovery(); -- Must return 't'

-- Check last replayed WAL position and time
SELECT pg_last_wal_replay_lsn() AS replay_lsn,
now() - pg_last_xact_replay_timestamp() AS replay_delay;
-- replay_delay shows how far behind the standby is
-- For log-shipping, expect delay up to archive_timeout + transfer time
```

STREAMING REPLICATION (PG 16/17): PG 16/17 — Verify streaming replication health:

Streaming Replication Verification (PG 16/17):

```
-- On PRIMARY (PG 16/17): Check streaming replication + slots
SELECT pid, username, application_name, client_addr, state,
sent_lsn, write_lsn, flush_lsn, replay_lsn,
pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS lag,
sync_state
FROM pg_stat_replication;

-- Check replication slot status
SELECT slot_name, slot_type, active, restart_lsn,
pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained_wal
FROM pg_replication_slots;
```

Standby Verification (PG 16/17):

```
-- On STANDBY (PG 16/17): Check WAL receiver
SELECT pg_is_in_recovery(); -- Must return 't'
SELECT status, received_lsn, received_tli, slot_name
FROM pg_stat_wal_receiver;
```

Step 2: Enable wal_log_hints (If Not Already ON)

```
-- Check current setting
SHOW wal_log_hints;

-- If 'off', enable it (REQUIRES RESTART)
ALTER SYSTEM SET wal_log_hints = on;
sudo systemctl restart postgresql-14 -- or postgresql-16, etc.

-- Verify
SHOW wal_log_hints; -- Must return 'on'
```

WARNING: Enabling `wal_log_hints` requires a restart. Plan during a maintenance window. **This must be done BEFORE any failover — cannot be fixed retroactively.**

5. Phase 2 – Execute Failover

Purpose: The primary becomes unavailable; promote the secondary to maintain service.

Step 3: Primary Server Fails / Is Stopped

```
# On Primary Server (192.168.1.10) – applies to all PG versions
sudo systemctl stop postgresql-<version>

# OR using pg_ctl
pg_ctl stop -D /apps/pgsql_data/<version> -m fast
```

Shutdown Mode	Behavior	When to Use
smart	Waits for all clients to disconnect	Planned maintenance
fast	Terminates connections, writes checkpoint, clean shutdown	Normal failover simulation
immediate	Crash-like, no checkpoint	Emergency only — needs recovery on restart

Step 4: Promote Secondary to New Primary

Before promoting: Verify the standby has replayed all available WAL.

LOG-SHIPPING (PG 13/14): PG 13/14 — Before promoting: Ensure all archived WAL files have been restored. Check that no new WAL files remain in the archive that have not yet been replayed. The standby may be behind by up to `archive_timeout` seconds of data that was not yet archived when the primary went down.

Pre-Promotion — Log-Shipping (PG 13/14):

```
# On Standby (PG 13/14): Check replay position
psql -U postgres -c "SELECT pg_last_wal_replay_lsn(),
now() - pg_last_xact_replay_timestamp() AS delay;"

# If primary was stopped cleanly, manually copy any remaining WAL:
# (from old primary's pg_wal to archive, then let standby replay)
scp postgres@192.168.1.10:/apps/pgsql_data/14/pg_wal/0000000100* \
/apps/pgsql_archives/
```

STREAMING REPLICATION (PG 16/17): PG 16/17 — Before promoting: Verify the WAL receiver has caught up.

Pre-Promotion — Streaming (PG 16/17):

```
-- On Standby (PG 16/17): Verify WAL fully replayed
SELECT pg_last_wal_receive_lsn() = pg_last_wal_replay_lsn() AS caught_up;
```

Promote (All Versions):

```
# Method 1: pg_ctl (all versions)
pg_ctl promote -D /apps/pgsql_data/<version>

# Method 2: SQL function (PG 12+)
psql -U postgres -c "SELECT pg_promote(wait => true, timeout => 60);"

# Verify promotion succeeded
psql -U postgres -c "SELECT pg_is_in_recovery();"
-- Must return 'f' (false) – server is now a primary

# Confirm new timeline
psql -U postgres -c "SELECT timeline_id, checkpoint_lsn FROM pg_control_checkpoint();"
-- timeline_id should be 2 (incremented from 1)
```

Step 5: Simulate Timeline Divergence (Optional — Testing)

```
-- On New Primary – create test data to verify pg_rewind later
CREATE TABLE IF NOT EXISTS test_divergence (
  id SERIAL PRIMARY KEY, data TEXT, created_at TIMESTAMP DEFAULT now());
INSERT INTO test_divergence (data, created_at)
SELECT 'Post-promotion row ' || i, now() FROM generate_series(1, 1000) AS i;
```

6. Phase 3 – Prepare Old Primary for Reintegration

Step 6: Understand Why Direct Restart Is Dangerous

CRITICAL: NEVER start the old primary as a standalone server after the standby has been promoted. It is on Timeline 1 while the new primary is on Timeline 2 — starting it creates a **split-brain** condition with unrecoverable data loss.

Step 7: Ensure Old Primary Is Cleanly Shut Down

```
# On Old Primary (192.168.1.10)
pg_ctl stop -D /apps/pgsql_data/<version> -m fast

# Verify it is stopped
pg_ctl status -D /apps/pgsql_data/<version>
# Expected: pg_ctl: no server running

# Verify clean shutdown
pg_controldata /apps/pgsql_data/<version> | grep 'Database cluster state'
# Must show: 'shut down'
```

If the primary crashed (not shut down cleanly):

```
# Start briefly for crash recovery (block all app connections first!)
pg_ctl start -D /apps/pgsql_data/<version>
sleep 20 # Wait for crash recovery to complete
pg_ctl stop -D /apps/pgsql_data/<version> -m fast

# Verify clean state
pg_controldata /apps/pgsql_data/<version> | grep 'Database cluster state'
```

Step 8: Verify Timeline Divergence

```
# On Old Primary (stopped)
pg_controldata /apps/pgsql_data/<version> | grep -E 'Timeline|checkpoint|state'
# Expected: TimelineID: 1, state: shut down

# On New Primary (running)
psql -U postgres -c "SELECT timeline_id, checkpoint_lsn FROM pg_control_checkpoint();"
# Expected: timeline_id: 2
```

Step 8B: Back Up Old Primary PGDATA (Production Safety)

CRITICAL: In production, **always** back up the old primary's data directory before running pg_rewind. If pg_rewind fails midway, the data directory may be left unusable.

```
# Option 1: LVM snapshot (fastest)
lvcreate --snapshot --name pgdata_snap --size 50G /dev/vg_data/lv_pgdata

# Option 2: rsync
rsync -a --delete /apps/pgsql_data/<version>/ \
/apps/backups/pre_rewind_$(date +%Y%m%d)/

# Option 3: tar
tar czf /apps/backups/pre_rewind_$(date +%Y%m%d).tar.gz \
-C /apps/pgsql_data/<version> .
```

7. Phase 4 – Execute pg_rewind

STREAMING REPLICATION (PG 16/17): PG 16/17 only — Step 9: Create a replication slot on the new primary **before** starting the old primary as a standby.

Step 9: Create Replication Slot (PG 16/17 Streaming Only)

```
-- On New Primary (192.168.1.20) – PG 16/17 ONLY
SELECT pg_create_physical_replication_slot('old_primary_slot');

-- Verify
SELECT slot_name, slot_type, active, restart_lsn
FROM pg_replication_slots;
```

LOG-SHIPPING (PG 13/14): PG 13/14: Skip Step 9 — log-shipping does not use replication slots. WAL retention is managed entirely through the archive directory.

Step 10: Run pg_rewind on Old Primary (All Versions)

The `pg_rewind` command itself is **identical** regardless of whether the cluster uses log-shipping or streaming replication. It connects to the new primary's live instance to determine the divergence point.

Option 1: Source-server connection (Recommended)

```
# On Old Primary (192.168.1.10) – ALL versions
pg_rewind \
--target-pgdata=/apps/pgsql_data/<version> \
--source-server="host=192.168.1.20 port=5432 user=postgres \
dbname=postgres connect_timeout=30" \
--progress \
--verbose
```

Option 2: With WAL archive fetching (if missing WAL errors)

```
# --restore-target-wal (-c) uses the target's restore_command to
# retrieve any WAL files needed during the rewind.
pg_rewind \
--target-pgdata=/apps/pgsql_data/<version> \
--source-server="host=192.168.1.20 port=5432 user=postgres dbname=postgres" \
--restore-target-wal \
--progress --verbose
```

Option 3: Dry-run (PG 13+ — preview without changes)

```
pg_rewind \
--target-pgdata=/apps/pgsql_data/<version> \
--source-server="host=192.168.1.20 port=5432 user=postgres dbname=postgres" \
--dry-run --progress
```

Option 4: --write-recovery-conf (PG 13+ streaming)

```
# Automatically creates standby.signal + writes primary_conninfo
# Useful for PG 16/17 streaming. You still need to add slot name and password.
pg_rewind \
--target-pgdata=/apps/pgsql_data/<version> \
--source-server="host=192.168.1.20 port=5432 user=postgres dbname=postgres" \
--write-recovery-conf \
--progress --verbose
```

Expected Successful Output:

```
connected to server
servers diverged at WAL location 0/3000060 on timeline 1
rewinding from last common checkpoint at 0/3000028 on timeline 1
reading source file list
reading target file list
reading WAL in target
need to copy 1234 MB (total source directory size is 45678 MB)
1234/1234 MB (100%) copied
creating backup label and updating control file
syncing target data directory
Done!
```

CRITICAL: `pg_rewind` copies `postgresql.auto.conf` from the source (new primary) to the target (old primary). This **overwrites** the old primary's `auto.conf` with the new primary's settings. In Step 11, you must **review and edit** this file — do not blindly append.

pg_rewind Time Estimates:

Database Size	Typical Divergence	pg_rewind	pg_basebackup
< 10 GB	< 100 MB	1 – 3 min	5 – 15 min
10 – 100 GB	< 1 GB	3 – 15 min	30 min – 2 hr
100 GB – 1 TB	< 10 GB	10 – 45 min	3 – 12 hr
> 1 TB	< 50 GB	30 min – 2 hr	12 – 48 hr

8. Phase 5 – Configure Old Primary as New Standby

Purpose: Configure the rewound old primary as a standby. The configuration differs significantly between log-shipping and streaming replication.

WARNING: pg_rewind copied the new primary's `postgresql.auto.conf` to the old primary. **First review** the current contents — it may contain stale `primary_conninfo` entries from when the new primary was itself a standby. Remove stale entries before writing new configuration.

Step 11: Review and Edit `postgresql.auto.conf`

```
# On Old Primary (192.168.1.10) – ALL VERSIONS

# Step 11a: Create standby.signal (PG 12+)
touch /apps/pgsql_data/<version>/standby.signal

# Step 11b: Review the overwritten postgresql.auto.conf
cat /apps/pgsql_data/<version>/postgresql.auto.conf
# LOOK FOR: stale primary_conninfo, primary_slot_name entries
# REMOVE any that point back to the old primary (192.168.1.10)
```

LOG-SHIPPING (PG 13/14): PG 13/14 — Log-Shipping standby configuration. No `primary_conninfo` or replication slot is needed. The standby retrieves WAL entirely from the archive via `restore_command`.

Log-Shipping Configuration (PG 13/14):

```
# Step 11c-LS: Write log-shipping standby configuration
# On Old Primary (now standby) – PG 13 / PG 14

cat >> /apps/pgsql_data/14/postgresql.auto.conf <<'EOF'

# =====
# Log-Shipping Standby Configuration – After pg_rewind
# =====

# Retrieve archived WAL files from shared archive directory
restore_command = 'cp /apps/pgsql_archives/%f %p'

# Clean up archived WAL files no longer needed by this standby
archive_cleanup_command = 'pg_archivecleanup /apps/pgsql_archives %r'

# Follow the latest timeline (auto-follows timeline switches)
recovery_target_timeline = 'latest'

EOF
```

NOTE: For log-shipping, the standby polls the archive directory for new WAL files. There is no direct TCP connection to the primary. The replication lag depends on how quickly WAL files are archived and the standby's polling interval.

STREAMING REPLICATION (PG 16/17): PG 16/17 — Streaming replication standby configuration. Requires `primary_conninfo` with the connection string to the new primary, and `primary_slot_name` referencing the slot created in Step 9.

Streaming Replication Configuration (PG 16/17):

```
# Step 11c-SR: Write streaming standby configuration
# On Old Primary (now standby) – PG 16 / PG 17

cat >> /apps/pgsql_data/16/postgresql.auto.conf <<'EOF'

# =====
# Streaming Replication Standby Config – After pg_rewind
# =====

# Direct streaming connection to the new primary
primary_conninfo = 'host=192.168.1.20 port=5432 user=replicator password=YOUR_PASSWORD
application_name=old_primary_standby'

# Replication slot to prevent WAL recycling on new primary
primary_slot_name = 'old_primary_slot'

# Follow latest timeline
recovery_target_timeline = 'latest'

# Fallback: if streaming is interrupted, fetch from archive
restore_command = 'cp /apps/pgsql_archives/%f %p'
archive_cleanup_command = 'pg_archivecleanup /apps/pgsql_archives %r'

EOF
```

Parameter Reference:

Parameter	Log-Shipping (PG 13/14)	Streaming (PG 16/17)
<code>primary_conninfo</code>	Not used	Required — connection string to new primary
<code>primary_slot_name</code>	Not used	Required — slot name on new primary
<code>restore_command</code>	Required — sole WAL source	Optional but recommended as fallback
<code>archive_cleanup_command</code>	Recommended	Recommended
<code>recovery_target_timeline</code>	'latest'	'latest'
<code>standby.signal</code>	Required (PG 12+)	Required (PG 12+)

For PostgreSQL 13 (standby.signal + auto.conf applies). For PG 11 and earlier:


```
# PG 11 and earlier use recovery.conf instead of standby.signal
cat > /apps/pgsql_data/11/recovery.conf <<'EOF'
standby_mode = 'on'
restore_command = 'cp /apps/pgsql_archives/%f %p'
recovery_target_timeline = 'latest'
EOF
```

Step 12: Set Correct Ownership and Permissions

```
# All versions – on old primary
chown postgres:postgres /apps/pgsql_data/<version>/standby.signal
chown postgres:postgres /apps/pgsql_data/<version>/postgresql.auto.conf
chmod 700 /apps/pgsql_data/<version>

# Verify
ls -la /apps/pgsql_data/<version>/ | grep -E 'standby.signal|postgresql.auto.conf'
```

TIP: PostgreSQL refuses to start if data directory permissions exceed 700. This is a common post-pg_rewind startup failure, especially when backup/copy operations changed ownership.

Step 12B: Verify Replication User (PG 16/17 Streaming Only)

```
-- On New Primary (192.168.1.20) – PG 16/17 only
SELECT rolname, rolreplication, rolcanlogin
FROM pg_roles WHERE rolname = 'replicator';

-- If not present, create it:
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'your_secure_password';
```

9. Phase 6 – Start and Verify Replication

Step 13: Start Old Primary as New Standby

```
# On Old Primary (192.168.1.10) – all versions
sudo systemctl start postgresql-<version>

# Monitor startup logs
tail -f /apps/logs/postgresql-*.log
```

Critical Log Messages:

Log Message	Meaning	Status
LOG: entering standby mode	Server started in standby mode	Good
LOG: redo starts at 0/3000028	WAL replay begins from divergence	Good
LOG: consistent recovery state reached	Data is consistent	Good
LOG: started streaming WAL from primary on timeline 2	Streaming connected (PG 16/17)	Good
LOG: restored log file "000000020000..." from archive	Archive restore working (PG 13/14)	Good
LOG: database system is ready to accept read-only connections	Standby operational	Good

Step 14: Verify Replication from Both Sides

LOG-SHIPPING (PG 13/14): PG 13/14 — Log-shipping verification:

Log-Shipping Verification (PG 13/14):

```
-- On NEW PRIMARY (192.168.1.20) – PG 13/14
-- Verify archiver is running (WAL files being created for the standby)
SELECT archived_count, last_archived_wal, last_archived_time
FROM pg_stat_archiver;

-- On NEW STANDBY (192.168.1.10) – PG 13/14
SELECT pg_is_in_recovery(); -- Must return 't'

-- Check replay position and delay
SELECT pg_last_wal_replay_lsn() AS replay_lsn,
now() - pg_last_xact_replay_timestamp() AS replay_delay;

-- For log-shipping, some delay (seconds to minutes) is normal
-- depending on archive_timeout and WAL activity
```

STREAMING REPLICATION (PG 16/17): PG 16/17 — Streaming verification:**Streaming Verification on Primary (PG 16/17):**

```
-- On NEW PRIMARY (192.168.1.20) — PG 16/17
SELECT application_name, client_addr, state,
pg_wal_lsn_diff(sent_lsn, replay_lsn) AS lag_bytes,
pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS lag,
write_lag, flush_lag, replay_lag, sync_state
FROM pg_stat_replication;
-- state = 'streaming', lag = '0 bytes' = healthy
```

Streaming Verification on Standby (PG 16/17):

```
-- On NEW STANDBY (192.168.1.10) — PG 16/17
SELECT pg_is_in_recovery(); -- Must return 't'
SELECT status, received_lsn, received_tli, slot_name
FROM pg_stat_wal_receiver;
-- status = 'streaming', received_tli = 2
```

Step 15: Verify Data Consistency

```
-- Run on BOTH servers (results must match after replication catches up)
SELECT count(*) AS total_rows FROM test_divergence;

-- Advanced checksum verification
SELECT md5(string_agg(id::text || data, '' ORDER BY id))
FROM test_divergence;
```

10. Phase 7 – Cleanup and Post-Recovery Tasks

Step 16: Monitor Replication Health

LOG-SHIPPING (PG 13/14): PG 13/14 — Monitor archive health: Ensure WAL files are being archived on the new primary and restored on the standby. Watch for archive failures.

Archive Monitoring (PG 13/14):

```
-- On New Primary (PG 13/14): Check archiver
SELECT archived_count, failed_count, last_archived_wal, last_failed_wal
FROM pg_stat_archiver;

-- Check archive directory growth
du -sh /apps/pgsql_archives/
ls -ltr /apps/pgsql_archives/ | tail -5
```

STREAMING REPLICATION (PG 16/17): PG 16/17 — Monitor replication slots:

Slot Monitoring (PG 16/17):

```
-- On New Primary (PG 16/17): Check slot health
SELECT slot_name, active, active_pid,
pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained_wal
FROM pg_replication_slots;
```

Condition	Meaning	Action
active = t, retained_wal < 1 GB	Healthy	No action
active = f, retained_wal < 1 GB	Standby temporarily offline	Monitor
active = f, retained_wal > 10 GB	Standby down, WAL accumulating	Investigate now
active = f, retained_wal > 50 GB	Disk space at risk	Consider dropping slot

Step 17: Drop Stale Replication Slots

When the old primary was active (PG 16/17 only), it had replication slots for its standbys. Now that it is a standby, those old slots are stale.

```
-- On New Standby (Old Primary) – PG 16/17 only
-- Step 1: Confirm you are on the standby
SELECT pg_is_in_recovery(); -- Must return 't'

-- Step 2: List slots
SELECT slot_name, active FROM pg_replication_slots;

-- Step 3: Drop stale slots (only if active = f)
SELECT pg_drop_replication_slot('standby_1');
```

CRITICAL: Only drop slots on the **standby** (old primary). **NEVER** drop `old_primary_slot` on the new primary — it is in active use.

Step 18: Post-Recovery Checklist

1. **Verify archive_command on new primary:** WAL archiving must be active for both log-shipping standbys and as a safety net for streaming setups
2. **Configure archive on new primary (PG 13/14):** If the new primary was previously a standby, it may not have `archive_command` set. Enable it now to keep serving WAL to the standby
3. **Update connection pooler:** Point PgBouncer/pgpool-II/HAProxy to 192.168.1.20 as primary
4. **Update application connection strings:** Use DNS/VIP to avoid manual changes in future
5. **Verify backup jobs:** `pg_dump` / `pg_basebackup` / `pgBackRest` should target the new primary
6. **Update monitoring dashboards:** Reflect new primary/standby topology
7. **Synchronous replication (PG 16/17):** If configured, update `synchronous_standby_names` on the new primary
8. **Document the event:** Record timestamp, cause, duration, data loss, steps taken

11. Troubleshooting Guide

Error: "could not find previous WAL record"

```
pg_rewind: error: could not find previous WAL record at 0/3000028
```

Cause: Required WAL files recycled or archived from the target.

Solutions:

- Use `--restore-target-wal (-c)` to fetch WAL from archive
- Manually copy the WAL file from archive to `$PGDATA/pg_wal/`
- Ensure `restore_command` is configured in the target's `postgresql.conf`

Error: "target server must be shut down cleanly"

```
# Solution: start for crash recovery, then stop cleanly
pg_ctl start -D /apps/pgsql_data/<version>
sleep 20
pg_ctl stop -D /apps/pgsql_data/<version> -m fast
# Then retry pg_rewind
```

WARNING: When starting the old primary for crash recovery, block all application access. It will start on the OLD timeline.

Error: "wal_log_hints must be enabled"

CRITICAL: This error **cannot be fixed retroactively**. You must rebuild with `pg_basebackup`. Enable `wal_log_hints = on` on all clusters proactively.

Error: Connection refused / authentication failed

```
# Verify the new primary is running
pg_isready -h 192.168.1.20 -p 5432

# Test connectivity
psql "host=192.168.1.20 port=5432 user=postgres dbname=postgres" -c "SELECT 1;"

# Check pg_hba.conf for old primary's IP
grep '192.168.1.10' /apps/pgsql_data/<version>/pg_hba.conf

# Check firewall (AlmaLinux)
sudo firewall-cmd --list-ports
```

Error: "source and target on same timeline"

Cause: No divergence — `pg_rewind` is unnecessary. Simply configure the old primary as a standby (Steps 11–12) and start it.

Log-Shipping Specific: Standby Not Catching Up (PG 13/14)

Log-Shipping Troubleshooting (PG 13/14):

```
# Check if restore_command is working
# Test manually:
cp /apps/pgsql_archives/0000000200000000000000003 /tmp/test_wal

# Check standby logs for restore errors
grep -i 'restore_command\|could not\|error' /apps/logs/postgresql-*.log | tail -20

# Verify archive directory permissions
ls -ld /apps/pgsql_archives/
# Must be readable by postgres user on standby
```

Streaming Specific: Replication Slot Not Found (PG 16/17)

Streaming Troubleshooting (PG 16/17):

```
# FATAL: replication slot "old_primary_slot" does not exist
# Solution: Create the slot on the new primary
psql -U postgres -h 192.168.1.20 -c \
"SELECT pg_create_physical_replication_slot('old_primary_slot');"
```

Tablespace Considerations (All Versions)

pg_rewind handles tablespaces correctly only if directory paths are identical on both servers. If paths differ, use `--tablespace-mapping` (PG 15+) or create matching symlinks before pg_rewind.

12. Automation Script

Two script variants are provided: one for log-shipping (PG 13/14) and one for streaming (PG 16/17). The core `pg_rewind` step is identical; only the standby configuration differs.

Common Header (Both Versions):

```
#!/bin/bash
set -euo pipefail

# --- CONFIGURATION ---
OLD_PRIMARY_IP="192.168.1.10"
NEW_PRIMARY_IP="192.168.1.20"
PG_VERSION="14" # Change: 13, 14, 16, or 17
PGDATA="/apps/pgsql_data/${PG_VERSION}"
ARCHIVE_DIR="${PGDATA}/archive"
POSTGRES_USER="postgres"
LOG_FILE="/apps/logs/pg_rewind_$(date +%Y%m%d_%H%M%S).log"
BACKUP_DIR="/apps/backups/pre_rewind_$(date +%Y%m%d)"

log() { echo "[$(date +%Y-%m-%d %H:%M:%S)] [$1] $2" | tee -a "$LOG_FILE"; }
die() { log "ERROR" "$1"; exit 1; }
ssh_cmd() { ssh -o ConnectTimeout=10 "${POSTGRES_USER}@$1" "$2"; }

# Pre-flight + Stop + Backup + Rewind (common)
ssh_cmd "$OLD_PRIMARY_IP" "pg_ctl stop -D $PGDATA -m fast" 2>/dev/null || true
ssh_cmd "$OLD_PRIMARY_IP" "mkdir -p $BACKUP_DIR && rsync -a $PGDATA/ $BACKUP_DIR/"
ssh_cmd "$OLD_PRIMARY_IP" "pg_rewind --target-pgdata=$PGDATA \
--source-server='host=$NEW_PRIMARY_IP port=5432 user=$POSTGRES_USER \
dbname=postgres' --restore-target-wal --progress --verbose"
```

Log-Shipping Configuration Block (PG 13/14):

```
# --- STANDBY CONFIG: LOG-SHIPPING (PG 13/14) ---
ssh_cmd "$OLD_PRIMARY_IP" "touch $PGDATA/standby.signal"
ssh_cmd "$OLD_PRIMARY_IP" "cat >> $PGDATA/postgresql.auto.conf <<EOF
restore_command = 'cp ${ARCHIVE_DIR}/%f %p'
archive_cleanup_command = 'pg_archivecleanup ${ARCHIVE_DIR} %r'
recovery_target_timeline = 'latest'
EOF"
```

Streaming Configuration Block (PG 16/17):


```
# --- STANDBY CONFIG: STREAMING (PG 16/17) ---
REPLICATOR_USER="replicator"
REPLICATOR_PASSWORD="your_secure_password"
SLOT_NAME="old_primary_slot"

# Create replication slot on new primary
ssh_cmd "$NEW_PRIMARY_IP" "psql -U $POSTGRES_USER -c \
\"SELECT pg_create_physical_replication_slot('${SLOT_NAME}');\""

ssh_cmd "$OLD_PRIMARY_IP" "touch $PGDATA/standby.signal"
ssh_cmd "$OLD_PRIMARY_IP" "cat >> $PGDATA/postgresql.auto.conf <<EOF
primary_conninfo = 'host=$NEW_PRIMARY_IP port=5432 user=$REPLICATOR_USER \
password=$REPLICATOR_PASSWORD application_name=old_primary_standby'
primary_slot_name = '${SLOT_NAME}'
recovery_target_timeline = 'latest'
restore_command = 'cp ${ARCHIVE_DIR}/%f %p'
EOF"
```

Common Footer (Both Versions):

```
# --- START + VERIFY (common) ---
ssh_cmd "$OLD_PRIMARY_IP" "chown -R postgres:postgres $PGDATA/standby.signal \
$PGDATA/postgresql.auto.conf && chmod 700 $PGDATA"
ssh_cmd "$OLD_PRIMARY_IP" "pg_ctl start -D $PGDATA" ; sleep 15
RECOVERY=$(ssh_cmd "$OLD_PRIMARY_IP" \
"psql -U $POSTGRES_USER -t -c 'SELECT pg_is_in_recovery();'" | xargs)
[ "$RECOVERY" = "t" ] || die "NOT in recovery mode!"
log "INFO" "=== pg_rewind completed successfully ==="
```

13. Production Best Practices

Proactive Configuration

- Enable `wal_log_hints = on` on **all** clusters at deployment time — zero-cost insurance for `pg_rewind`
- Always configure `archive_mode = on` and a working `archive_command`, even for streaming replication. The archive serves as a WAL safety net and is required by `pg_rewind`'s `--restore-target-wal` option
- For PG 13/14 log-shipping: set `archive_timeout = 300` (5 min) to limit the maximum data loss window
- For PG 16/17 streaming: set `max_slot_wal_keep_size = 50GB` to prevent runaway disk usage from stale slots
- Use a dedicated replication user (PG 16/17) — never use postgres superuser for replication
- Use DNS/VIP for primary addressing to eliminate manual app changes during failover

During Failover

- Verify standby WAL replay is caught up before promoting
- For PG 13/14: manually copy any remaining WAL from old primary's `pg_wal` to the archive before promoting
- For PG 16/17: create the replication slot on new primary immediately after promotion
- Always back up old primary's PGDATA before running `pg_rewind`
- Use `--dry-run` (PG 13+) to preview `pg_rewind` before committing

Log-Shipping Specific Practices (PG 13/14)

- Monitor the archive directory size — old WAL files should be cleaned by `archive_cleanup_command` on the standby
- After failover, ensure the **new primary** has `archive_command` configured. It was previously a standby and may not have this set
- Test `restore_command` independently before relying on it during `pg_rewind` or failover
- Consider a cron job to verify archive integrity (compare `pg_stat_archiver` counts vs. directory listing)

Streaming Specific Practices (PG 16/17)

- Monitor replication slots aggressively — an inactive slot with growing WAL retention is a disk-space emergency
- Set `max_slot_wal_keep_size` to cap slot WAL retention (slot becomes invalid when exceeded)
- Keep PostgreSQL minor versions consistent between primary and standby

postgresql.auto.conf Safety Rule

CRITICAL: On a standby server, **never** use `ALTER SYSTEM RESET ALL`. This wipes `postgresql.auto.conf` entirely, including `primary_conninfo` and `restore_command`, breaking replication immediately. Always use `ALTER SYSTEM RESET parameter_name` for individual parameters.

Replication-Sensitive Parameters (Update Standby First)

Parameter	Why Standby-First	Restart?
max_connections	Standby needs >= primary value for transaction replay	Yes
max_worker_processes	Standby needs >= primary value for parallel replay	Yes
max_prepared_transactions	Standby needs >= primary value for 2PC	Yes
max_locks_per_transaction	Standby needs >= primary value for lock replay	Yes

14. Quick Reference Checklist

Condensed checklist for actual failover events. Differences between log-shipping and streaming are marked with **[LS]** and **[SR]**.

Phase	Step	Action	Applies To
Pre-Failover	1	Verify replication health (pg_stat_archiver [LS] / pg_stat_replication [SR])	Both
Pre-Failover	2	Confirm wal_log_hints = on, full_page_writes = on, archive_mode = on	Both
Failover	3	Primary fails / is stopped	Both
Failover	4-LS	[LS] Copy remaining WAL from old primary pg_wal to archive	PG 13/14
Failover	4	Verify standby WAL replay, then promote	Both
Prepare	7	Ensure old primary is cleanly shut down (pg_controldata: shut down)	Both
Prepare	8	Verify timeline divergence (TimelineID differs)	Both
Prepare	8B	Back up old primary PGDATA	Both
pg_rewind	9	[SR only] Create replication slot on new primary	PG 16/17
pg_rewind	10	Run pg_rewind (--dry-run first)	Both
Configure	11	Create standby.signal + edit postgresql.auto.conf	Both
Configure	11-LS	[LS] Set restore_command + recovery_target_timeline	PG 13/14
Configure	11-SR	[SR] Set primary_conninfo + primary_slot_name + recovery_target_timeline	PG 16/17
Configure	12	Set ownership (postgres:postgres) and mode (700)	Both
Verify	13	Start standby, watch logs	Both
Verify	14	Confirm replication: pg_stat_archiver [LS] / pg_stat_replication [SR]	Both
Verify	15	Verify data consistency	Both
Cleanup	16	Monitor archive health [LS] / slot health [SR]	Both
Cleanup	17	[SR only] Drop stale slots on old primary	PG 16/17
Cleanup	18	Update pooler, apps, monitoring, backups, docs	Both

End of Runbook